THE HIDDEN ALGEBRA OF THE AES ENCRYPTION STANDARD

G. AVERKOV

AES and its different views on the bytes. *AES, the advanced encryption standard,* is the cryptosystem based on the parameterized encryption function, which we denote

$$\mathrm{AES}_k : V \to V.$$

Our function is parameterized by $k \in V$. The message space, they key space and the cipher-text space are all the same set V, which is a $\mathbb{Z}_2$-vector space of dimension 128. Plainly speaking, the message $m \in V$, the key $k \in V$, the cipher text $c = \mathrm{AES}_k(v) \in V$ all carry 128 bits of information.

The AES encryption function is defined as a composition of a sequence of functions. In other words, we have a sequence of operations, applied one after another, which bring us from the message in V to a cipher text in V over intermediate values that also belong to V. As a diagram, it looks like this

$$V \to V \to \cdots \to V.$$

When we do computations in V within this "assembly line", in order to evaluate AES_k, we interpret V in different ways. At the most elementary level, we can view V as just $V \hat{=} \mathbb{Z}_2^{128}$. With this interpretations, the elements of V are just tuples consisting of 128 elements of $\mathbb{Z}_2$. A somewhat more structured interpretation at the next level is $V \hat{=} \mathbb{Z}_2^{8\times4\times4}$. With this interpretation, elements of V are three-dimensional arrays of size $8 \times 4 \times 4$ over $\mathbb{Z}_2$. If you like, you can think about 4×4 table, with each entry storing a list of 8 bits. Strictly speaking $\mathbb{Z}_2^{128}$ and $\mathbb{Z}_2^{8\times4\times4}$ are not identical sets, but they are the "same set" in the sense that one can just list elements of the $8 \times 4 \times 4$ array in some order in order to obtain a list of 128 elements. Thus, $\mathbb{Z}_2^{128}$ and $\mathbb{Z}^2 8 \times 4 \times 4$ are isomorphic $\mathbb{Z}_2$-vector spaces, with some isomorphism $\mathbb{Z}_2^{128} \leftrightarrow {\mathbb{Z}_2}^{8\times4\times4}$. Nevertheless, in order to avoid technicalities we want to suppress this identification and just denote all the different but isomorphic vector spaces involved into the description of the AES by the same letter V. This is what is called an abuse of notation in math. That is, we tentatively write things done not absolutely formally correct, but we write them in a way that is interpretable correctly. So, the writing is somewhat inaccurate, but the math is correct. From the computer-science perspective, you might think about this as an implicit conversion.

Let us go on and have the other relevant interpretations fixed. Viewing ${\mathbb{Z}_2}^{8\times4\times4}$ as $({\mathbb{Z}_2}^8)^{4\times4}$, the matrix of 8-dimensional vectors, we can identify ${\mathbb{Z}_2}^8$ with different 8-dimensional $\mathbb{Z}_2$-algebras. We have the isomorphisms

$$F \longleftrightarrow {\mathbb{Z}_2}^8 \longleftrightarrow G,$$

where $F = R/f$ and $G = R/g$ the $\mathbb{Z}_2$-algebras obtained by taking the quotient of $R = \mathbb{Z}_2[t]$ modulo $f = t^8 + t^4 + t^3 + t + 1$ and $g = t^8 + 1$, respectively. We will call elements of F, G and ${\mathbb{Z}_2}^8$ are *bytes*, as they indeed carry eight bits of information. Still, do not forget that bytes are not just bytes in our context, since we attach a structure of a vector space or a ring to them. Our byte has three interpretations:

an 8-tuple of the elements of $\mathbb{Z}_2$, an element of the ring $F = R/f$ and an element of the ring $G = R/g$. The polynomial $f \in R$ is irreducible. Hence, F is the Galois field of size $2^8 = 256$. In contrast, $g \in R$ is not irreducible, and thus G is a unitary commutative ring but not a field.

No matter, which interpretation of a byte we pick in what follows, a byte will always be a vector over $\mathbb{Z}_2$. Accordingly to the interpretation of a byte, the space V may be given different interpretations. It can be viewed as ${\mathbb{Z}_2}^{4\times 4}$ or ${\mathbb{Z}_2}^{16}$, as $F^{4\times 4}$ or F^{16}, as $G^{4\times 4}$ or G^{16}.

SubBytes. We have introduced different interpretations of the set of all bytes, with the most elementary one being ${\mathbb{Z}_2}^8$. The heart of the AES is the SubBytes operation, which we denote

$$\mathrm{SB} : {\mathbb{Z}_2}^8 \to {\mathbb{Z}_2}^8.$$

It is the only operation involved in the AES that is not affine over $\mathbb{Z}_2$. That is, SubBytes is the only difficult genuinely non-linear operation. SubBytes is arising as a composition of several operations. The respective diagram is

$${\mathbb{Z}_2}^8 \longleftrightarrow F \longrightarrow F \longleftrightarrow {\mathbb{Z}_2}^8 \longleftrightarrow G \longrightarrow G \longleftrightarrow {\mathbb{Z}_2}^8.$$

Here, the double arrows correspond to the interpretation part, $F \to F$ is the operation, which we would like to call the "safe inversion" and $G \to G$ is a certain affine map on the ring G. First, let us discuss the "safe inversion", which is merely $\mathrm{inv} : F \to F$, given by

$$\mathrm{inv}(x) := \begin{cases} x^{-1}, & x \in F \setminus \{0\}, \\ 0, & x = 0. \end{cases}$$

As we know, another way to describe inv is

$$\mathrm{inv}(x) = x^{|F|-2} = x^{254},$$

because $x^{|F|} = x$ for each $x \in F$. The safe inversion is inverse to itself, meaning that $\mathrm{inv}(\mathrm{inv}(x)) = x$.

The operation $G \to G$, which we would like to denote as $T : G \to G$ is an affine transformation $T(x) = ax + b$, where

$$a = [1 + t + t^2 + t^3 + t^4]_g$$
$$b = [1 + t + t^5 + t^6]_g.$$

The operation T is invertible with the inverse $T^{-1}(x) = a^{-1}(x - b)$. This is clear, because a is invertible, while the invertibility of a is clear in view of $\gcd(1 + t + t^2 + t^3 + t^4, t^8 + 1) = \gcd(1 + t + t^2 + t^3 + t^4, (t + 1)^8) = 1$. The fact that T is related to rings is usually hidden the description of the AES, because a typical gives T through its linear representation as the $\mathbb{Z}_2$-linear map ${\mathbb{Z}_2}^8 \to {\mathbb{Z}_2}^8$

$$\begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \\ x_7 \end{bmatrix} \mapsto \begin{bmatrix} 1 & 0 & 0 & 0 & 1 & 1 & 1 & 1 \\ 1 & 1 & 0 & 0 & 0 & 1 & 1 & 1 \\ 1 & 1 & 1 & 0 & 0 & 0 & 1 & 1 \\ 1 & 1 & 1 & 1 & 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 & 0 & 0 & 0 \\ 0 & 1 & 1 & 1 & 1 & 1 & 0 & 0 \\ 0 & 0 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 0 & 0 & 1 & 1 & 1 & 1 & 1 \end{bmatrix} \begin{bmatrix} x_0 \\ x_1 \\ x_2 \\ x_3 \\ x_4 \\ x_5 \\ x_6 \\ x_7 \end{bmatrix} + \begin{bmatrix} 1 \\ 1 \\ 0 \\ 0 \\ 0 \\ 1 \\ 1 \\ 0 \end{bmatrix}.$$

This is a valid description, but it hides the algebra behind the operation. When we combine both operations, we get the SubBytes,

$$\mathrm{SB}(x) = T(\mathrm{inv}(x)).$$

In this description, we do not mention the conversion from one interpretation of bytes to the other one, to keep the notation simple. If we mentioned the conversion there would be conversion maps (vector space isomorphism) between the application of T and inv and after an application of T. The SubBytes operation can now be applied in componentwise fahsion to any number of bytes. When we have N bytes which we transform via $(\mathbb{Z}_2{}^8)^N \to (\mathbb{Z}_2{}^8)^N$ in this way, we denote the respective operation $\mathrm{SB}^{\oplus N}$. The operation is given by $\mathrm{SB}^{\oplus N}(b_1, \ldots, b_N) = (\mathrm{SB}(b_1), \ldots, \mathrm{SB}(b_N))$. We call it the N-fold direct sum of SB. In most of the cases, one would call both SB and $\mathrm{SB}^{\oplus N}$ SubBytes. In the AES the 16-fold direction sum of SubBytes is applied to $V \hat{=} (\mathbb{Z}_2{}^8)^{4\times 4}$. In other words, each 16 bytes of the 4×4 matrix of bytes are modified using the SubBytes transformation. As one can see, the operation $\mathrm{SB}^{\oplus 16}$ is non-linear by local to bytes: the 8 bits of the output of SB depend non-linear on the 8 bits on the 8 bits of the input of SB. The objective of the other transformations is to distribute this non-linearity globally so that the 128 bits of the cipher text of the AES depend non-linearly on *all* of the 128 bits of the plain text of the AES.

Shift Rows, permutations in groups of 4 bytes. At the level of bytes, the shift rows operation $\mathrm{SR} : (\mathbb{Z}_2{}^8)^{4\times 4} \to (\mathbb{Z}_2{}^8)^{4\times 4}$ is the operation that sends the 4×4 array of bytes $b = (b_{i,j})_{\substack{i=0,\ldots,3\\ j=0,\ldots,3}}$ ot an array of bytes $b' = (b'_{i,j})_{\substack{i=0,\ldots,3\\ j=0,\ldots,3}}$ with the property that $b'_i = (b'_{i,0}, \ldots, b'_{i,3})$ is the left shift of $b_i = (b_{i,0}, \ldots, b_{i,3})$ by i positions. This means, $b'_{i,j} = b_{i,(j+i) \bmod 4}$. This operation is $\mathbb{Z}_2$-linear and it is more global than the SubBytes but it is not the full-scale global, because it just permutations bytes within each single row of the 4×4 matrix of bytes, independently.

Interestingly, this operation can be explained through an application of the ring isomorphism. We need a ring of bi-variate polynomials for that purpose. Let us consider the ring $\mathbb{Z}_2[x, y]$ and let us take the quotient of this ring modulo the ideal $I = \langle x^4 - 1, y^4 - 1\rangle$. Then the elements of the quotient ring $H := \mathbb{Z}_2[x, y]/I$ can be represented as the cosets of the polynomials $h(x, y) = \sum_{i,j=0,\ldots,3} c_{i,j} x^i y^j$ modulo I. In H we can fix the $\mathbb{Z}_2$-basis consisting of the 16 cosets $m_{i,j} = [x^i y^j]_I$ with $0 \le i, j \le 3$. The shift-row operation corresponds to sending $m_{i,j}$ to $m_{i,(j-i) \bmod 3}$. This can also be written as $m_{i,j} \mapsto m_{i,(j+3i) \bmod 3}$ and can be interpreted as application of the ring homomorphism $H \to H$ induced by the substitution of the monomials according through the map $x^i y^j \mapsto x^i y^{j+3i} = (xy^3)^i y^j$. Hence, $\mathrm{SR}([h(x, y)]_I) = h(xy^3, y)]_I$ is a ring homomorphism, which is invertible since $\mathrm{SR}^{-1}([h(x, y)]_I) = [h(xy, y)]_I$. In order to see that the ring homomorphism $\mathbb{Z}_2[x, y] \to \mathbb{Z}_2[x, y]$ given by $f(x, y) \mapsto f(xy^3, y)$ induces a well-defined ring homomorphism SR as above, it suffices to check that $h(xy^3, y) \in I$ whenever $h \in I$. This is not hard to see since the generators x^4-1, y^4-1 get mapped to $(xy^3)^4 - 1$ and $y^4 - 1$, respectively, both congruent to 0 mod I.

From the above ring-isomorphism perspective, the shift rows is $\mathrm{SR} = \mathrm{SR}^{\oplus 8}$, as SR, when use the interpretation

$$V \hat{=} H^8.$$

Mix columns. We first see how to mix one column. Let us take an element of F^4, a column of 4 bytes, and interpret F^4 as

$$P = F[z]/p.$$

where $p = z^4+1$. As F has characteristic two, we have $p(z) = (z+1)^4$. We have the interpretation of V as $F^{4\times 4}$, which we interpret as P^4, but interpreting each column $(c_0, \dots, c_3)$ of a matrix in $F^{4\times 4}$ as $[c_0 + c_1 z + c_2 z^2 + c_3 z^3]_p \in F$.

As F has characteristic two, we still have $z^4 + 1 = (z+1)^4$ in $F[z]$, as we had in the case of $\mathbb{Z}_2[t]$. Let $\bar{t} = [t]_f \in F$ and consider the mix-column operation MC, $\mathrm{MC}(x) = ux$, where $u = [\bar{t} + z + z^2 + (\bar{t}+1)z^3]_p$. This map, which is linear over P and it is invertible, since $u \in F$ is invertible. So, the inverse of this map is $\mathrm{MC}^{-1}(x) = u^{-1}x$. The mix-columns operation is the operation 4-fold direct sum of MC with itself $\mathrm{MC}^{\oplus 4} : P^4 \to P^4$. When shift-rows and mix-columns are applied one after another (as will be the case in the description that follows), one establishes a global inter-dependence among the 16 bytes of the matrices from $V \hat{=} ({\mathbb{Z}_2}^8)^{4\times 4}$. Indeed, after the shift of rows, the four bytes of every single column get distributed each into its own column so that each of these four bytes takes part in its own mixture of columns.

Both mix-columns and shift-rows are $\mathbb{Z}_2$-affine, so that their composition is also a $\mathbb{Z}_2$-affine map.

Round keys. The key information $k \in V$ enters into the rounds of the AES encryption in a non-trivial fashion. Let us interpret view $V \hat{=} F^{4\times 4}$ as $V \hat{=} (F^4)^4$. That means, an element of V is a 4×4 matrix of bytes, which we can view as a list (b_0, b_1, b_2, b_3) of four columns $b_i \in F^4$, each being a four-element vector of bytes. We introduce the map $\mathrm{RK}_s(b_0, b_1, b_2, b_3) = (b'_0, b'_1, b'_2, b'_3)$ by the following rule:

$$\begin{aligned} b'_0 &= b_0 + \mathrm{SB}^{\oplus 4}(b_3) + s, \\ b'_1 &= b_1 + b'_0, \\ b'_2 &= b_2 + b'_1, \\ b'_3 &= b_3 + b'_2, \end{aligned}$$

where $s \in K^4$. We can spell this out as

$$\mathrm{RK}_s(b_0, b_1, b_2, b_3) = (b_0 + w, b_0 + b_1 + w, b_0 + b_1 + b_2 + w, b_0 + b_1 + b_2 + b_3 + w),$$

where $w = \mathrm{SB}^{\oplus 4}(b_3) + s$. Let us call this map the round map $\mathrm{RK}_s : V \to V$ with the shift parameter s.

Complete AES. Start with $k, v \in V$. And then do the following:

- $k^{(0)} = k$ and $v^{(0)} = v + k^{(0)}$
- For $i = 1, \dots, 10$:
- $k^{(i)} = \mathrm{RK}_{s^{(i)}}(k^{(i-1)})$, where $s^{(i)} = (\bar{t}^i, 0, 0, 0)$.
- $v^{(i)} = k^{(i)} + \mathrm{MC}^{\oplus 4}(\mathrm{SR}^{\oplus 8}(\mathrm{SB}^{\oplus 16}(v^{(i-1)})))$ if $i \in \{1, \dots, 9\}$.
- $v^{(i)} = k^{(i)} + \mathrm{SR}^{\oplus 8}(\mathrm{SB}^{\oplus 16}(v^{(i-1)}))$ if $i = 10$.

The output is $\mathrm{AES}_k(v) = v^{(10)}$. There are modification to this procedure where the number of rounds is different. What matters is that SB is locally non-linear, but this linearity is spread to the global scope by using the more global operations like mix-columns and shift-rows. The rounds $i = 0$ and $i = 10$ are special. For $i = 0$, just the key k is taken and the added to the plain text v. In the round $i = 10$, the the mix-column operation is skipped. The map $\mathrm{AES}_k(v)$ depends non-linearly on both v and k, because SB is applied on both k and v iteratively. The elements $k^{(i)} \in V$ are called round keys.

What is so special about the non-linearity of the sub-bytes? The sub-bytes operation is very far from being an $\mathbb{Z}_2$-affine map. Assume that we have a vector space V over a field K and we consider an affine map $T : V \to V$, where $T(v) = L(v) + s$ and $L : V \to V$ is linear. If T is affine then $T(v + a) - T(v) = L(v + a) - L(v) = L(v) + L(a) - L(v) = L(a)$. That is, the number of times vector $(s, L(s))$ is present in the graph of T is $|V|$, the size of V. This suggests the metric

$$\Delta(T) := \max_{a \in V \setminus \{0\},\ b \in V} |\{v \in V : T(v + a) - T(v) = b\}|,$$

called differential non-uniformity. If the $\Delta(T) \leq |K|^\ell$, the maximum dimension of the vector subspace W of V, on which the restriction $T|_W$ is K-affine is ℓ. It turns out that $\Delta(\text{inv}) = 4$. That means, since that inv is only $\mathbb{Z}_2$-affine on a two-dimensional subspace. It is not hard to show that $\Delta(\text{inv}) \leq 4$. The fact that $\Delta(\text{inv}) = 4$ has to do with....

Decryption for AES. Calculate all round keys based on k and do computations of $v^{(10)}, \ldots, v^{(0)}$ in this order.

Implementation. AES boils down to arithmetic in $\mathbb{Z}_2$, which basically only requires AND and XOR, which are directly available at the software and hardware level. The SubBytes function is stored as a lookup table so that the operations involved in the definition SubBytes need not be re-calculated each time AES is used. Lookup tables are called S-boxes in the context of symmetric cryptography. So, the SubBytes S-box stores 256 bytes, one for each possible input of the SubBytes operation.